Build Journey: Roadblocks and Resolutions

Minhal Imam

1. Build Overview

Scope: A complete equity pairs trading pipeline — from initial commit to CI-green repo with honest statistical findings — built over roughly two days of focused work. The pipeline ingests 30 tickers of daily OHLCV data, runs Engle-Granger cointegration tests on 54 within-sector pairs, generates z-score trading signals, backtests the strategy, and produces automated data quality reports and charts.

Timeline:

- **Hour 0–2:** Initial project structure. Full module scaffold (ingest, reconcile, cointegration, signals, backtest), requirements, .gitignore, main.py orchestrator. First commit: acdad63.
- **Hour 2–4:** README with methodology docs, run script (run_pipeline.sh), dependency fixes for macOS. Commits 5e5575e, a21c6b7.
- **Hour 4–6:** First full pipeline run. Generated charts, backtest results, cointegration heatmap. Updated README with actual results. Commits 32fba24, 3d66542, f160efb.
- **Hour 6–8:** Added data investigation report and monitoring module (initially 8 checks). Set up GitHub Actions CI. Commits 29b39cb, 57d85fd, 02cf2cc, dd3a9c9.
- **Hour 8–12:** The rigor rewrite. Fixed monitor stubs, added BH-FDR correction, added out-of-sample validation, rewrote the investigation report and README with honest findings. Commits 03a87de through 02236ff.

2. Roadblocks and Resolutions

Alpha Vantage ingestion. The pipeline was designed with dual-source ingestion: yfinance as primary, Alpha Vantage as secondary for cross-source reconciliation. The AV free tier allows 5 API calls per minute and 25 per day. Ingesting 30 tickers would require spreading requests across multiple days. Rather than building brittle retry logic around a severely rate-limited free API, I decided to skip AV ingestion (--skip-av), document the decision, and keep the reconciliation module in place so it

works correctly when AV data is available. The investigation report diagnoses this from the actual evidence: the data/raw/alphavantage/ directory is empty, no API calls were made. This is documented as a known limitation.

Dependency version conflicts. The initial requirements.txt pinned exact versions (e.g., numpy==1.26.4). On macOS with Apple Silicon, several of these had no pre-built wheels and attempted to compile from source, which failed due to missing Fortran compilers and LAPACK headers. The fix was switching to flexible version pins (e.g., numpy>=1.24) that resolve to the latest pre-built wheel for the platform. Commit a21c6b7 captures this change. The lesson: exact pins are good for reproducibility in CI (where the platform is controlled) but can break local development on different architectures.

SSL certificate issues on macOS. Python's urllib3 failed to verify SSL certificates when calling yfinance, which uses requests internally. macOS ships its own certificate store that Python does not always pick up. The fix was running the Install Certificates.command script bundled with the macOS Python installer, which installs the certifi package's CA bundle. This is a well-known macOS Python gotcha but not obvious if you have not encountered it before.

Git lock file issues. During rapid iteration, a git operation was interrupted (likely a ctrl-C during a commit), leaving a .git/index.lock file that blocked subsequent git commands with "Unable to create index.lock: File exists." The fix was simply rm .git/index.lock. The underlying cause: interrupting git during an index write leaves a stale lock. Safe to remove manually because the lock is an advisory mutex, not data.

Cloud VM proxy restrictions. The initial plan was to run the pipeline in a cloud VM for reproducibility. However, the VM's network proxy blocked outbound connections to Yahoo Finance (yfinance's data source) and GitHub's authenticated endpoints. yf.download() returned empty DataFrames silently, and git push failed with authentication errors. The resolution was to run the pipeline locally via run_pipeline.sh, which worked because the local machine has unrestricted internet access. The VM was still useful for CI testing with synthetic fixtures.

CI fixture date issues. The initial test fixtures for the monitoring module used hardcoded dates ending "today" (the date the fixtures were created). When CI ran on a different day, the staleness check failed because the fixture data appeared stale. The fix (commit dd3a9c9) changed fixtures to use relative dates computed at test time, so the staleness check always evaluates against fresh-looking data. This is a common pattern in time-dependent test suites: never hardcode absolute dates in fixtures that test recency.

Monitor stub checks. After the initial build, one of the monitoring checks (dividend consistency) was effectively a stub: it always returned "pass" regardless of input, because the check logic was never implemented beyond the function signature. This was discovered during the rigor review and removed entirely (the check was not meaningful for the pipeline's scope), bringing the count from 8 to 7

real checks. The remaining checks were verified to fail correctly on bad data. Commit 03a87de.

3. The Rigor Rewrite

This is the most important part of the build.

After the first “complete” version of the pipeline, I stepped back and reviewed the project as if I were evaluating someone else’s work. The review surfaced several serious credibility issues.

No multiple testing correction. The pipeline tested 54 pairs at alpha = 0.05 and reported 9 as significant. But at 54 tests, the expected number of false positives is 2.7. Claiming 9 significant pairs without correcting for this is misleading. I added Benjamini-Hochberg FDR correction to src/cointegration.py (commit d65f6cf). Result: 0 of 9 survived. The raw significance was consistent with multiple testing noise.

No out-of-sample validation. The entire backtest used full-sample parameters — cointegration was tested and hedge ratios estimated on the same data used for trading. This is textbook overfitting. I built src/oos_validation.py (commit eaf1a10) to split the data at October 2023: 2 years training, 1 year test. The training-period Sharpe averaged 1.44; the test-period Sharpe averaged 0.15. Three of 4 pairs flipped return sign. The in-sample Sharpe was almost entirely an artifact of overfitting.

Post-hoc explanations stated as fact. The original investigation report explained why certain pairs worked (V/MA: “shared payment-processing exposure”) as though these were verified findings. They were not tested. I relabeled each as “Hypothesis: … Not tested in this report” and specified what analysis would be needed to verify (e.g., factor regression on XHB for HD/LOW’s housing-market hypothesis). Commit 1dd297c.

Stub monitoring check. A dividend consistency check in the monitor module always returned “pass.” Removed it and verified the remaining 7 checks against known-bad data.

Wrong holiday calendar. The investigation report stated an expected 749 NYSE trading days, which was an approximation. I verified against exchange_calendars.get_calendar(“XNYS”) and got the correct figure: 753 sessions. Commit fbe58c6.

The lesson: A project that reports a 1.5 Sharpe without caveats looks impressive until someone asks a hard question. A project that reports a Sharpe collapse honestly — and shows the work that uncovered it — is more credible and more interesting to talk about. The rigor rewrite turned a mediocre backtest showcase into a demonstration of the kind of critical thinking that matters in quantitative

work. This maps directly to what Jane Street values: willingness to find problems in your own data, say so, and build infrastructure that catches the next one.

4. Code Review Pass — Finding My Own Bugs

After squashing the commit history and cleaning up the repo for final submission, I did one more pass: a line-by-line code review of the two most logic-dense modules (signals.py and backtest.py), treating them as if they were someone else's pull request. I was looking for correctness bugs, not style nits. The review found five issues, three in the backtest and two in the signal generator.

Stop-loss re-entry bug (signals.py). The original stop-loss logic set position to zero when |z| >= 3, but the signal generator could immediately re-enter on the same bar because the entry condition was evaluated after the exit. This defeats the purpose of a stop-loss: the strategy exits at an extreme z-score and re-enters at that same extreme, guaranteeing a loss on the next mean-reversion move. The fix was a cooldown state machine: a locked_out flag is set on stop-out and cleared only when |z| drops back below the entry threshold (|z| < 2). This prevents re-entry until the spread has meaningfully reverted. I added two targeted tests (test_stoploss_cooldown_prevents_reentry and test_stoploss_cooldown_short_side) that verify the lockout on both long and short sides.

Empty DataFrame guard (signals.py). generate_signals() called sig_df["position"] unconditionally in main(), but when no pairs survive the significance filter, sig_df is an empty DataFrame and the column access raises a KeyError. Added an explicit if sig_df.empty guard before accessing columns.

Return normalization (backtest.py). The original backtest normalized daily P&L by dividing by the initial spread value (spread.iloc[0]). This is fragile: if the first spread value is near zero, returns blow up; if it's an outlier, all returns are scaled incorrectly. Replaced with median absolute spread, which is robust to outliers and gives a stable normalization base. The same fix was applied to the portfolio chart in report.py for consistency.

Portfolio aggregation (backtest.py). The original code computed portfolio returns by calling pct_change() on a cumulative P&L series, then taking the mean. This double-compounds: you compute cumulative P&L, differentiate it with pct_change (which divides by the running total, not the notional), then re-aggregate. The fix computes daily returns directly from spread-level P&L divided by the normalizer, then averages across pairs with nanmean and compounds once with cumprod. This matches how an equal-weight portfolio is actually constructed.

Win rate calculation (backtest.py). The original win rate was computed from a list of per-bar P&L values that were positive, which conflates "how many bars were profitable" with "how many round-trip trades were profitable." The fix uses vectorized trade identification: a trade starts when position goes from zero to

non-zero, gets a cumulative-sum ID, and the P&L is grouped by trade ID. Trades still open at the final bar are force-closed so they count. This gives a true round-trip win rate.

Impact on results: the corrected backtest numbers were identical to the pre-fix values for this particular dataset. The stop-loss cooldown never triggered because no pair hit |z| >= 3 and then immediately re-entered; the normalization change was negligible because median absolute spread was close to initial spread for all pairs; and all pairs shared the same 753-day date range, making the aggregation fix a no-op. The bugs were real — they would produce wrong numbers on different data — but the current dataset happened not to exercise them.

The lesson: identical output does not mean the code was correct. These were latent bugs: silent on this data, wrong on the next. The stop-loss re-entry bug, in particular, would have produced visibly wrong signals on any pair with a z-score excursion past ±3. Catching them before submission is better than discovering them in an interview when someone asks “what happens after a stop-out?”

5. What I Would Do Differently or Add Next

Point-in-time universe. The 30-ticker universe is survivorship-biased. A proper approach would use index membership lists as of each rebalancing date and include tickers that were later removed. This is the single largest methodological gap.

Rolling hedge ratios. The current static hedge ratio (estimated once, used everywhere) introduces lookahead bias. Re-estimating beta on a rolling basis (e.g., 60-day or 120-day) would make the backtest more realistic, though it would also reduce returns.

Transaction cost and borrow fee modeling. The backtest assumes zero costs. Adding realistic bid-ask spread estimates (5–10 bps per side for large-cap), commission costs, and short-borrow fees would show which pairs actually have a net positive edge.

Walk-forward validation. The current OOS test uses a single train/test split. A walk-forward framework with multiple rolling windows would give a more robust estimate of out-of-sample performance and reduce sensitivity to the choice of split date.

Live scheduled ingestion. Currently the pipeline runs as a one-shot batch job. Adding GitHub Actions or Airflow scheduling for daily ingestion and signal refresh would make it a continuously running system, which better demonstrates production infrastructure skills.